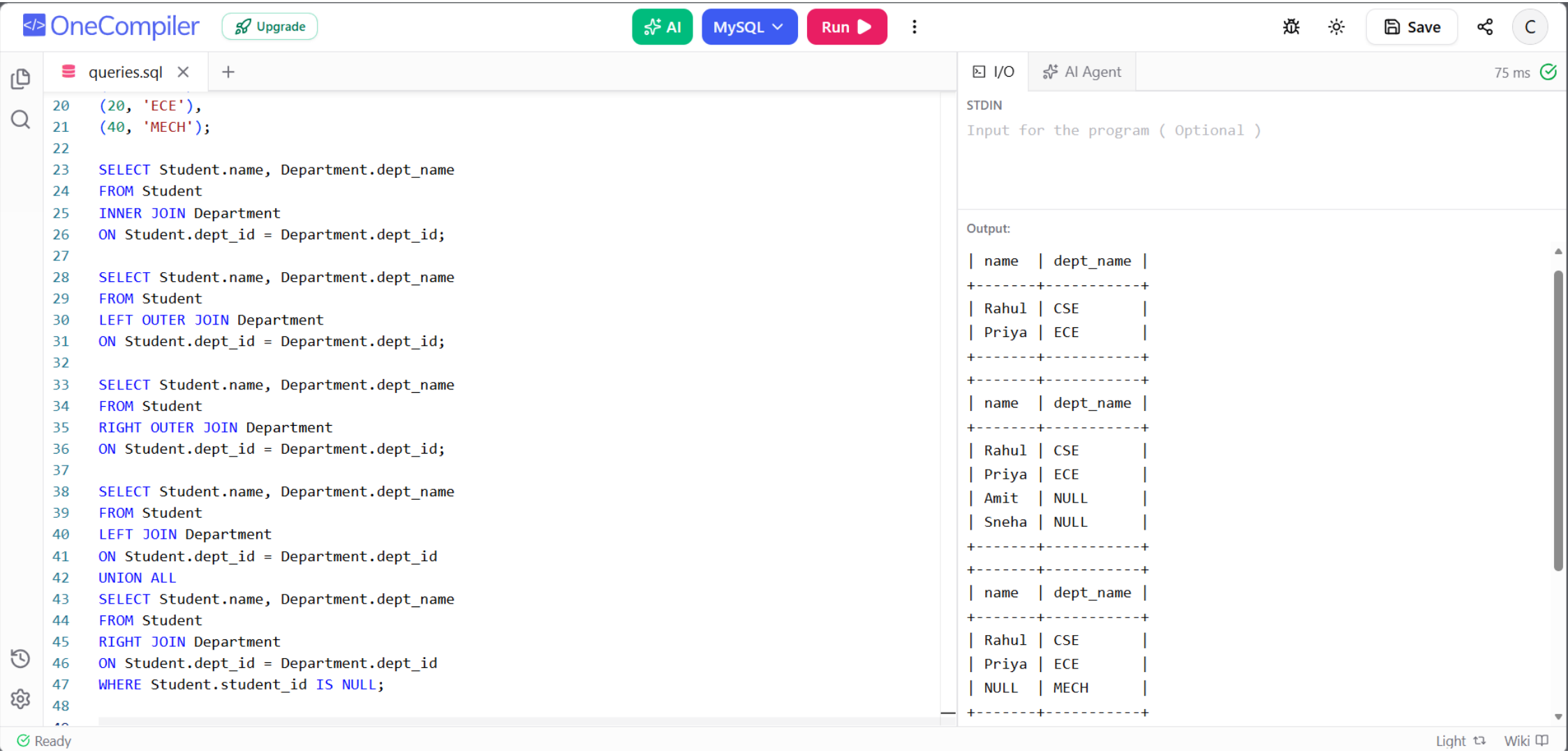
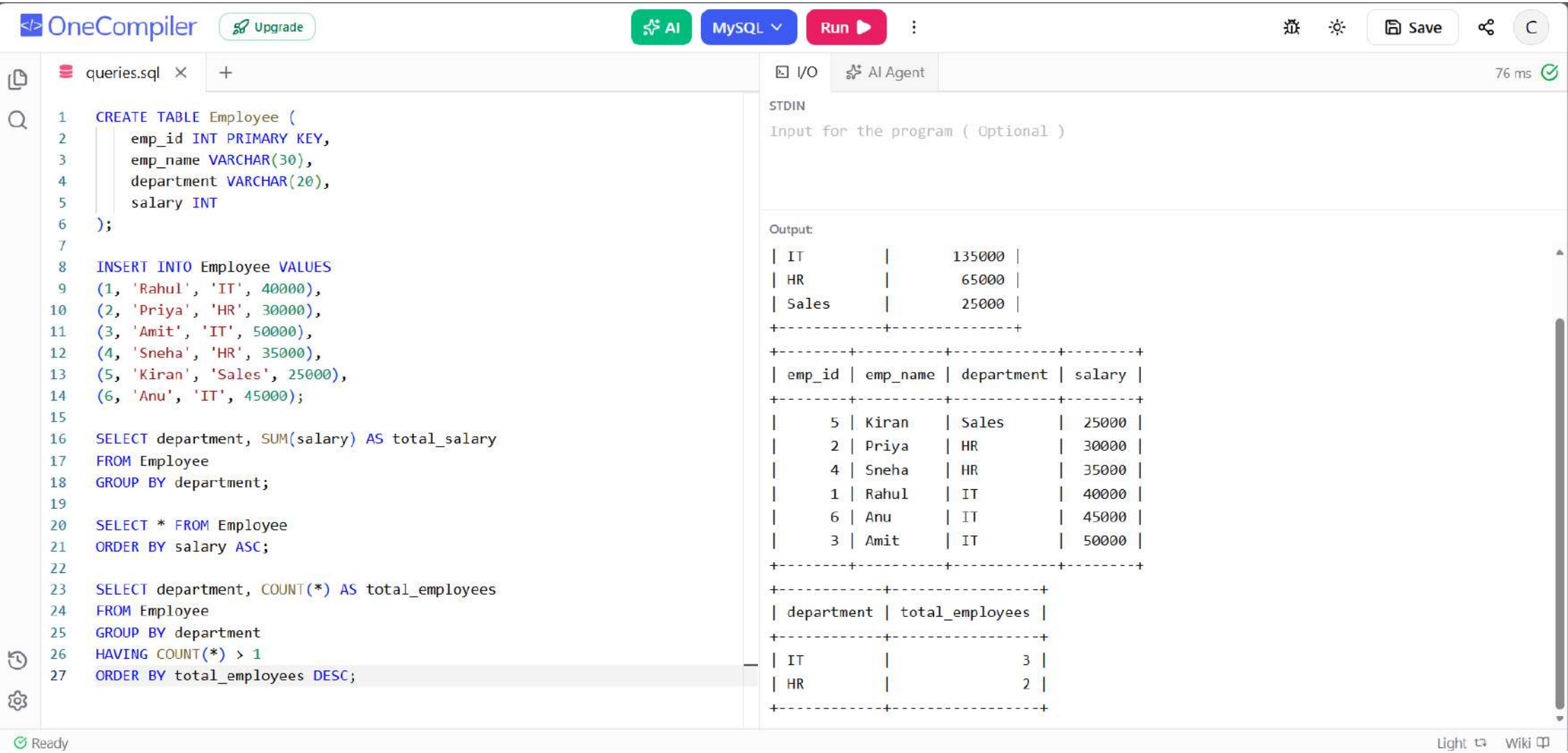

OneCompiler
Upgrade
AI
MySQL
Run
Save
C
queries.sql
(20, 'ECE'),
(40, 'MECH');
SELECT Student.name, Department.dept_name
FROM Student
INNER JOIN Department
ON Student.dept_id = Department.dept_id;
SELECT Student.name, Department.dept_name
FROM Student
LEFT OUTER JOIN Department
ON Student.dept_id = Department.dept_id;
SELECT Student.name, Department.dept_name
FROM Student
RIGHT OUTER JOIN Department
ON Student.dept_id = Department.dept_id;
SELECT Student.name, Department.dept_name
FROM Student
LEFT JOIN Department
ON Student.dept_id = Department.dept_id
UNION ALL
SELECT Student.name, Department.dept_name
FROM Student
RIGHT JOIN Department
ON Student.dept_id = Department.dept_id
WHERE Student.student_id IS NULL;
I/O
AI Agent
75 ms
STDIN
Input for the program (Optional)
Output:
| name | dept_name |
+-------+-----------+
| Rahul | CSE |
| Priya | ECE |
+-------+-----------+
+-------+-----------+
| name | dept_name |
+-------+-----------+
Rahul	CSE
Priya	ECE
Amit	NULL
Sneha	NULL
+-------+-----------+
+-------+-----------+
| name | dept_name |
+-------+-----------+
Rahul	CSE
Priya	ECE
NULL	MECH
+-------+-----------+
Ready
Light
Wiki




```sql
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(30),
    department VARCHAR(20),
    salary INT
);

INSERT INTO Employee VALUES
(1, 'Rahul', 'IT', 40000),
(2, 'Priya', 'HR', 30000),
(3, 'Amit', 'IT', 50000),
(4, 'Sneha', 'HR', 35000),
(5, 'Kiran', 'Sales', 25000),
(6, 'Anu', 'IT', 45000);

SELECT department, SUM(salary) AS total_salary
FROM Employee
GROUP BY department;

SELECT * FROM Employee
ORDER BY salary ASC;

SELECT department, COUNT(*) AS total_employees
FROM Employee
GROUP BY department
HAVING COUNT(*) > 1
ORDER BY total_employees DESC;
```

```
| IT         |       135000 |
| HR         |        65000 |
| Sales      |        25000 |
+------------+--------------+
+--------+----------+------------+--------+
| emp_id | emp_name | department | salary |
+--------+----------+------------+--------+
|      5 | Kiran    | Sales      |  25000 |
|      2 | Priya    | HR         |  30000 |
|      4 | Sneha    | HR         |  35000 |
|      1 | Rahul    | IT         |  40000 |
|      6 | Anu      | IT         |  45000 |
|      3 | Amit     | IT         |  50000 |
+--------+----------+------------+--------+
+------------+-----------------+
| department | total_employees |
+------------+-----------------+
| IT         |               3 |
| HR         |               2 |
+------------+-----------------+
```

queries.sql

```sql
DROP TABLE IF EXISTS student;

CREATE TABLE student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    dept VARCHAR(30),
    marks INT
);

INSERT INTO student VALUES (1,'Asha','CSE',82),(2,'Ravi','ECE',74);
DELIMITER //
CREATE PROCEDURE show_students()
BEGIN
    SELECT * FROM student;
END //
DELIMITER ;

CALL show_students();

DELIMITER //
CREATE PROCEDURE get_marks (IN sid INT, OUT m INT)
BEGIN
    SELECT marks INTO m FROM student WHERE id = sid;
END //
DELIMITER ;

CALL get_marks(1, @result);
SELECT @result;
```

I/O | AI Agent | 85 ms

STDIN

Input for the program (Optional)

Output:

```
|  1 | Asha | CSE  |    82 |
|  2 | Ravi | ECE  |    74 |
+----+------+------+-------+
+---------+
| @result |
+---------+
|      82 |
+---------+
+----------------+------------------------------------------------------------------------------
| Procedure      | sql_mode
+----------------+------------------------------------------------------------------------------
| insert_student | ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERR
    IN p_id INT, IN p_name VARCHAR(50), IN p_dept VARCHAR(30), IN p_marks INT
)
BEGIN
    INSERT INTO student (id, name, dept, marks)
    VALUES (p_id, p_name, p_dept, p_marks);
END | utf8mb4              | utf8mb4_0900_ai_ci   | utf8mb4_0900_ai_ci |
+----------------+------------------------------------------------------------------------------
```

OneCompiler Upgrade AI MySQL Run Save C

queries.sql

```
DELIMITER //
CREATE TRIGGER before_student_delete
BEFORE DELETE ON student
FOR EACH ROW
BEGIN
    IF OLD.marks > 90 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete a student with marks above 90';
    END IF;
END //
DELIMITER ;

DELIMITER //
CREATE TRIGGER after_student_delete
AFTER DELETE ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_log (action, stu_id, old_marks, new_marks)
    VALUES ('DELETE', OLD.id, OLD.marks, NULL);
END //
DELIMITER ;

-- Valid operations (all succeed and get logged)
INSERT INTO student VALUES (3, 'Meera', 'ISE', 91);
UPDATE student SET marks = 80 WHERE id = 2;
DELETE FROM student WHERE id = 2;

SELECT * FROM student;
SELECT * FROM student_log;
```

I/O AI Agent 91 ms

STDIN

Input for the program (Optional)

Output:

```
+----+-------+------+-------+
| id | name  | dept | marks |
+----+-------+------+-------+
|  1 | Asha  | CSE  |    82 |
|  3 | Meera | ISE  |    91 |
+----+-------+------+-------+
+--------+--------+--------+-----------+-----------+---------------------+
| log_id | action | stu_id | old_marks | new_marks | action_time         |
+--------+--------+--------+-----------+-----------+---------------------+
|      1 | INSERT |      3 |      NULL |        91 | 2026-10-09 13:58:34 |
|      2 | UPDATE |      2 |        74 |        80 | 2026-10-09 13:58:34 |
|      3 | DELETE |      2 |        80 |      NULL | 2026-10-09 13:58:34 |
+--------+--------+--------+-----------+-----------+---------------------+
```

Ready Light Wiki